AHONA SARKAR (CSE(IOT)_2ND YEAR) (12024052019002)

Design Patterm

Design Patterns Java Project Report

Section A: Architecture Overview

The application is a console-based Java project developed to demonstrate the implementation of multiple object-oriented design patterns. The patterns implemented include:

- Singleton Pattern (DatabaseConnection)
- Factory Pattern (ShapeFactory)
- Observer Pattern (WeatherData and DisplayDevice)
- Decorator Pattern (FeatureDecorator)
- Strategy Pattern (AdditionStrategy)
- Chain of Responsibility Pattern (Handler classes)

The main entry point of the application is Main.java, which initializes and tests all implemented design patterns.
The overall architecture follows modular object-oriented principles. Interfaces define common behaviors, while concrete classes implement specific functionality. This ensures loose coupling and high maintainability.
Control flow moves from the Main class to pattern-specific classes, which delegate behavior through interfaces and concrete implementations. State changes in the Observer pattern automatically trigger updates to registered observers.

Section B: Component and Class Selection

1. Interfaces (Shape, Observer, Strategy)

Purpose:
Define common behavior for multiple classes.
Why Appropriate:
Ensures polymorphism and loose coupling between components.
Limitation:
Interfaces cannot store object state directly.

2. Singleton (DatabaseConnection)

Purpose:
The Singleton pattern ensures that only one instance of a class exists and provides a global access point.
Example: A database connection manager where the application should maintain only one connection instance.
Why Appropriate:
Prevents multiple database connections from being created.
Limitation:
Introduces global state which may complicate testing.

3. Factory (ShapeFactory)

Purpose:
The Factory pattern is used to create objects without exposing the creation logic.
Example: A report generation system where different report types (PDF, Excel, HTML) are created depending on user input.
Why Appropriate:
Encapsulates object creation and improves flexibility.
Limitation:
Must be modified when new object types are added.

4. Observer (WeatherData & DisplayDevice)

Purpose:
Establishes a one-to-many relationship between objects.
Example: A stock market notification system where subscribers receive updates whenever stock prices change.
Why Appropriate:
Automatically updates observers when subject state changes.
Limitation:
Debugging may become complex with many observers.

5. Decorator (FeatureDecorator)

Purpose:
Adds new functionality dynamically to objects.
Example: A coffee ordering system where additional features such as milk or sugar are added dynamically to a base coffee object.
Why Appropriate:
Extends behavior without modifying original class.
Limitation:
Increases number of small classes.

6. Strategy (AdditionStrategy)
Purpose:
Encapsulates interchangeable algorithms.
Why Appropriate:
Removes complex conditional statements.
Limitation:
Increases number of classes.

7. Chain of Responsibility (Handlers)
Purpose:
Passes request through chain until handled.
Why Appropriate:
Reduces coupling between sender and receiver.
Limitation:
Requests may remain unhandled if chain is improperly configured.

Section C: Execution Flow and Interaction Logic

- getInstance() ensures only one Singleton object is created.
- getShape() in Factory selects and creates appropriate shape object.
- setTemperature() in Observer triggers notifyObservers().
- Decorator wraps original component and extends behavior.
- Strategy executes selected algorithm at runtime.
- Chain passes request through handlers until one processes it.

Data Flow:

- Input → Factory/Strategy → Object Created
- State Change → Subject → Observers Updated
- Request → Handler1 → Handler2 → Final Handling

Testing Approach:
Testing can be performed using unit tests to ensure that each pattern behaves correctly. For example, Singleton tests verify that only one instance is created, Factory tests ensure the correct object type is returned, and Observer tests confirm that observers receive updates when the subject state changes.

Section D: Validation and Error Handling

- Factory returns null for invalid shape type.
- Chain prints “Not handled” if no handler processes the request.
- Singleton prevents multiple object creation.

Validation ensures:

- Application stability
- Prevention of runtime errors
- Predictable behavior

Section E: Design Pattern Reasoning
The project demonstrates key object-oriented design principles through the use of well-known design patterns:

- Open/Closed Principle
- Separation of Concerns
- Loose Coupling
- Single Responsibility Principle

Each pattern was selected to demonstrate modular, scalable, and maintainable software design.

Section F: Data Handling and Persistence

- Observer list is stored using ArrayList.
- Objects are stored in memory during runtime.
- No permanent file or database persistence is implemented.

Consistency is maintained by automatic updates through the Observer pattern.

Section G: What-If Design Analysis

1. If all logic is written inside Main, the code becomes large and difficult to maintain.
2. If the application grows, database persistence and MVC architecture should be implemented.
3. Without Factory, object creation would require large conditional blocks.
4. Long-running tasks should use multithreading to maintain responsiveness.

Section H: Individual Extension

An additional shape, **Triangle**, was added to the Factory pattern without modifying existing client code.
This demonstrates:

- Extensibility
- Open/Closed Principle compliance

Section I: Debugging and Learning Reflection

Issues encountered:

- ClassNotFoundException due to incorrect project root selection.
- Incorrect use of named parameters.

Resolution:

- Corrected folder structure.
- Fixed Java syntax errors.

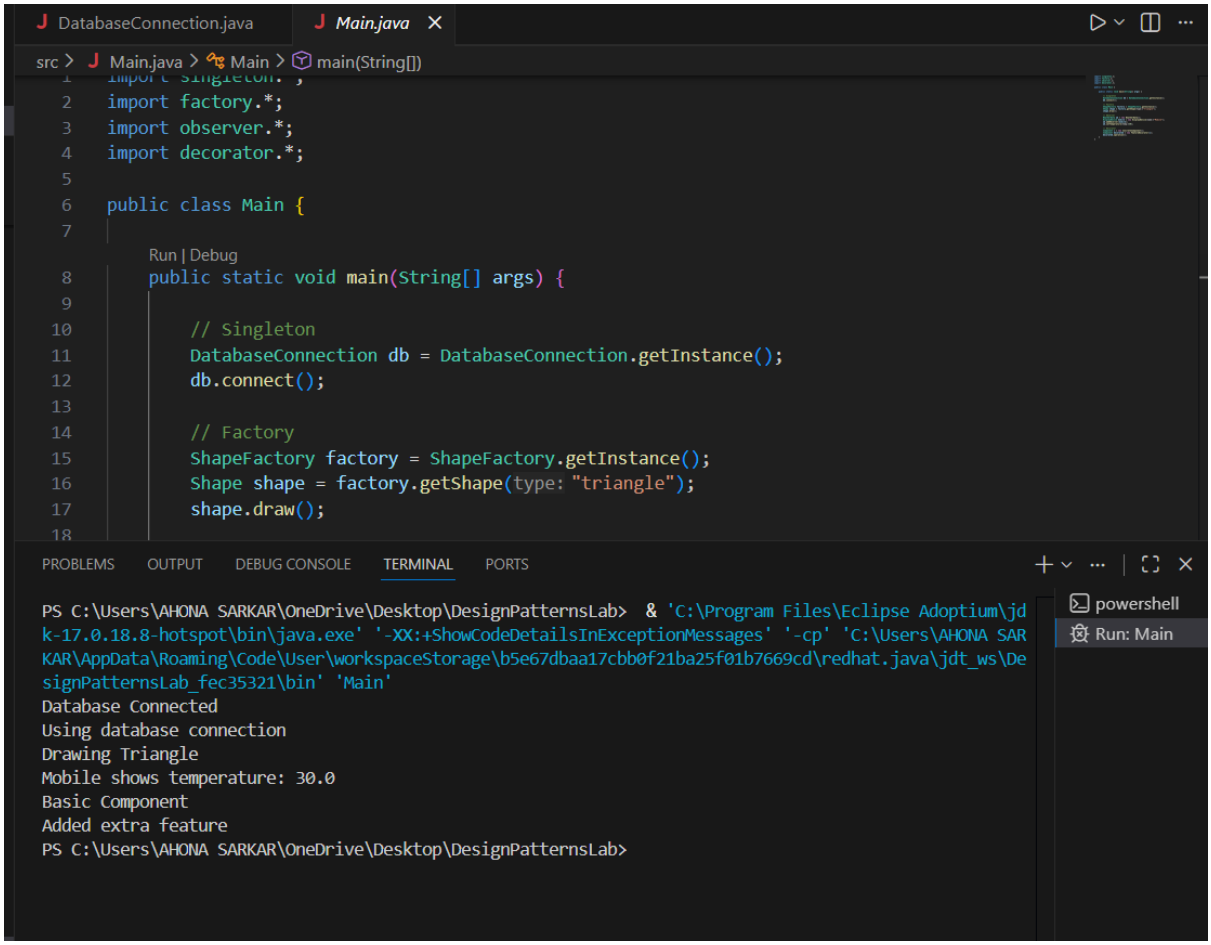
Conceptual understanding improved regarding:

- Package structure
- Object-oriented design principles
- Correct Java project execution

Section J: AI Usage Disclosure

AI assistance was used for structuring documentation and improving explanation clarity.
All code implementation, compilation, debugging, and testing were performed manually using VS Code and Java JDK.
Correctness was verified by successful compilation and execution of the program.

Output:


DatabaseConnection.java
Main.java
src > Main.java > Main > main(String[])
import factory.*;
import observer.*;
import decorator.*;
public class Main {
Run | Debug
public static void main(String[] args) {
// Singleton
DatabaseConnection db = DatabaseConnection.getInstance();
db.connect();
// Factory
ShapeFactory factory = ShapeFactory.getInstance();
Shape shape = factory.getShape(type: "triangle");
shape.draw();
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
powershell
Run: Main
PS C:\Users\AHONA SARKAR\OneDrive\Desktop\DesignPatternsLab> & 'C:\Program Files\Eclipse Adoptium\jdk-17.0.18.8-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\AHONA SARKAR\AppData\Roaming\Code\User\workspaceStorage\b5e67dbaa17cbb0f21ba25f01b7669cd\redhat.java\jdt_ws\DesignPatternsLab_fec35321\bin' 'Main'
Database Connected
Using database connection
Drawing Triangle
Mobile shows temperature: 30.0
Basic Component
Added extra feature
PS C:\Users\AHONA SARKAR\OneDrive\Desktop\DesignPatternsLab>